



Introduction to Data Science and Systems

Zaiqiao Meng
zaiqiao.meng@glasgow.ac.uk

Week 10 – Advanced Data
Systems

Outline

- SQL and NoSQL
- Graph Database Systems
 - Why Use Graph Data?
 - When to Use Graph Database?
- Vector Database Systems
 - Why Vector DBMS?
 - AKNN Search Algorithms

Relational Databases

Tabular Data

- Tabular data

columns = attributes for those observations

Rows = observations

Player	Minutes	Points	Rebounds	Assists
A	41	20	6	5
B	30	29	7	6
C	22	7	7	2
D	26	3	3	9
E	20	19	8	0
F	9	6	14	14
G	14	22	8	3
I	22	36	0	9
J	34	8	1	3

- SQL

```
SELECT *  
FROM   EMPLOYEE E, DEPARTMENT D  
WHERE  E.DNO = D.DNUMBER
```

Relational Databases

- The data is organized in tables. Tables are normalized for reduced data redundancy and better data integrity.
- Tables may have primary and foreign keys:
 - Primary Key is a minimal set of attributes (columns) that uniquely identifies a record (row) in a table.
 - Foreign Key establishes relationships between tables. It is a set of attributes in a table that refers to the primary key of another table.
- Query and transactions are coded using Standard Query Language (SQL).

Relational Databases - ACID

- **Atomicity:** Any transaction that updates multiple rows is treated as a single *unit*. A successful transaction performs *all* updates. A failed transaction performs *none* of the updates, i.e., the database is left unchanged.
- **Consistency:** Every transaction brings the database from one valid state to another. It guarantees to maintain all database invariants and constraints.
- **Isolation:** Concurrent execution of multiple transactions leaves the database in the same state as if the transactions were executed sequentially.
- **Durability:** Committed transactions are permanent, and survive even a system crash.

Relational Databases (RDBMS)

There are plenty to choose from:

- **Cloud Agnostic:** Oracle, Microsoft SQL Server, IBM DB2, [PostgreSQL](#), and [MySQL](#)
- **AWS:** Hosted PostgreSQL and MySQL in [Relational Database Service \(RDS\)](#)
- **Microsoft Azure:** Hosted SQL Server as [Azure SQL Database](#)
- **Google Cloud:** Hosted PostgreSQL and MySQL in [Cloud SQL](#), and also horizontally scaling [Cloud Spanner](#)

What's our new requirement?

- **Scalability** - Traditional SQL databases often struggle with horizontal scaling (adding more servers) because they use a single-node architecture for consistency and transactions.
- **Flexibility** - SQL databases enforce rigid schema designs. Any change in the schema (like adding new columns) requires updating the schema and potentially migrating data, which can be time-consuming and disruptive.
- **Handling Unstructured or Semi-Structured Data** - SQL databases are optimised for structured data stored in tabular formats, making them less efficient for handling unstructured or semi-structured data like docs.

What is NoSQL Database?

NoSQL databases cater to semi-structured data types: key-value, wide column, document (tree), graph or vector.

- A key-value store is a [dictionary or hash table](#) database. It is designed for CRUD operations with a unique key for each record:
 - Create(key, value): Add a key-value pair to the datastore
 - Read(key): Lookup the value associated with the key
 - Update(key, value): Change the existing value for the key
 - Delete(key): Delete the (key, value) record from the datastore
- The values do not have a fixed schema and can be anything from primitive values to compound structures. Key-value stores are highly partitionable (thus scale horizontally).
- [Redis](#) is a popular key-value store.

Difference between SQL and NoSQL

- **Data Model:** RDBMS databases are used for normalized structured (tabular) data strictly adhering to a relational schema. NoSQL datastores are used for non-relational data, e.g. key-value, document tree, graph.
- **Transaction Guarantees:** All RDBMS databases support ACID transactions, but most NoSQL datastores offer BASE transactions.
- **CAP Tradeoffs:** RDBMS databases prioritize strong consistency over everything else. But NoSQL datastores typically prioritize availability and partition tolerance (horizontal scale) and offer only [eventual consistency](#).

NoSQL Use Cases

In-memory Key-Value Data

- Same as a key-value store, but the data is in the memory instead of on the disk. It eliminates the disk IO overhead and serves as a fast cache.

Time Series Data

- A time series is a series of data points, indexed and ordered by timestamp. The timestamp is the key in the Time Series datastores.
- A time series can be modeled as:
 - **Key-value:** associated pairs of timestamps and values
 - **Wide column:** with the timestamp as the key for the table

NoSQL Use Cases

Geospatial Data

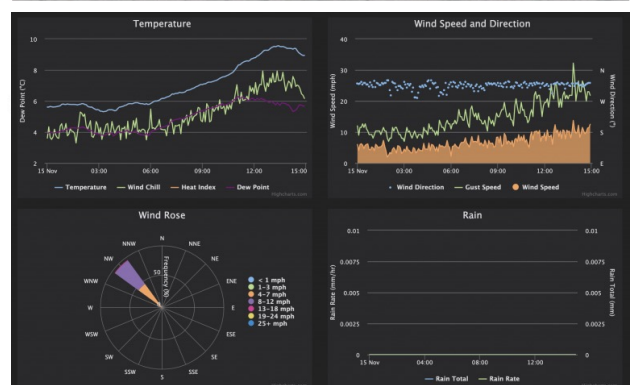
- A Geospatial database is a database to store geographic data (such as countries, cities, etc.). It is optimized for geospatial queries and geometric operations.

Text Search Data

- Text search on unstructured (natural) or semi-structured text is a common operation in many applications. The text can either be plain or rich (e.g. PDF), stored in a document database, or stored in a blob store. [Elastic Search](#) has been a popular solution.

Example: sailing weather station

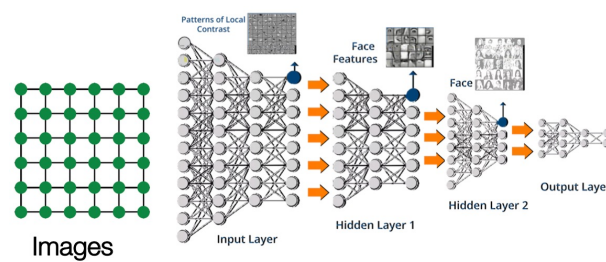
- **Task:** to measure local atmospheric conditions during voyages
- **Input:** wind speed, temperature, humidity, sunshine hours, etc., over 10,000 measurements
- **Systems:** InfluxDB, TimescaleDB, Prometheus



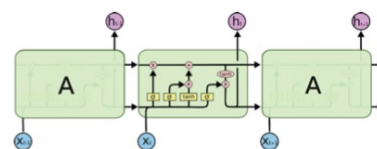
Graph Database Systems

Modern ML data structures

- Images

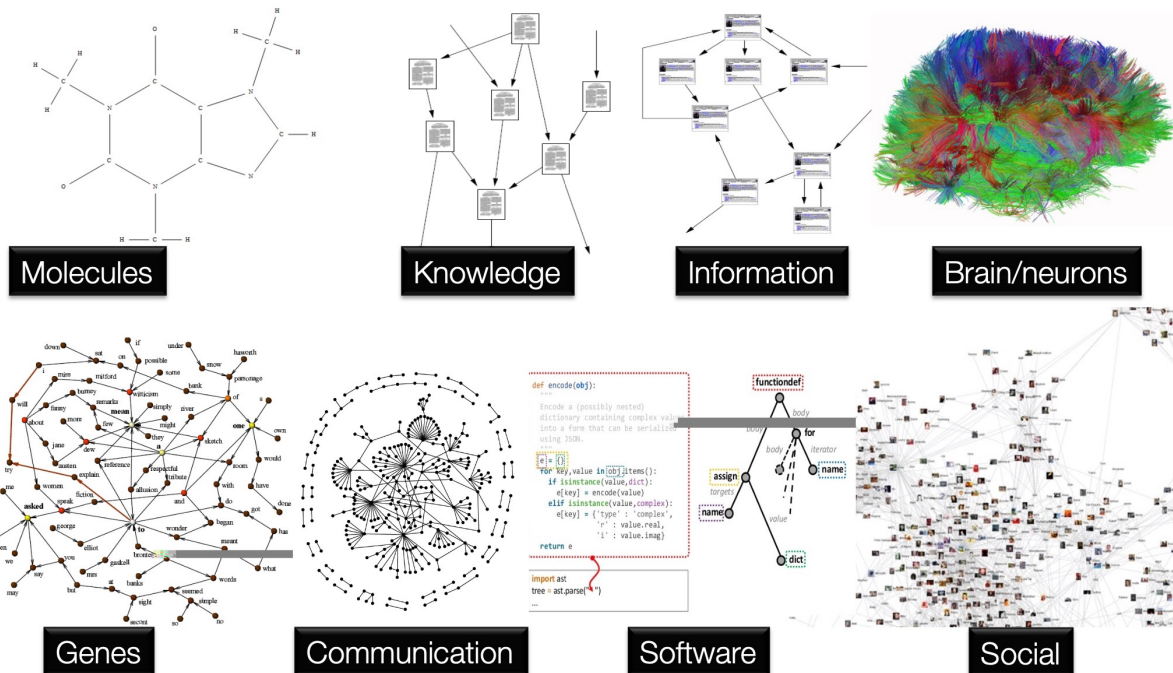


- Text/Speech



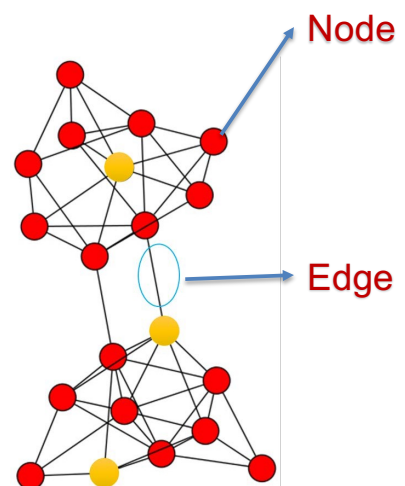
The modern deep learning “toolbox” is mostly designed for sequences & grids!

Graphs around us!



What is a graph?

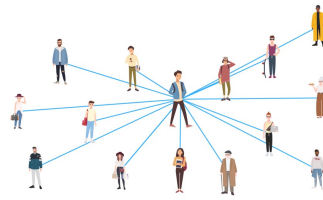
- **$G = (V, E)$**
 - **V** is a set of nodes/vertices
 - **E** is a set of edges/links/relations
 - Edges can be directed or undirected
- **Nodes (e.g. movies) can have features**
 - e.g., node types, attributes, neighbours
- **Edges can have features**
 - e.g., edge types (e.g. “acted”, “directed”), edge weights



Different types of graphs

- **Homogeneous graphs**

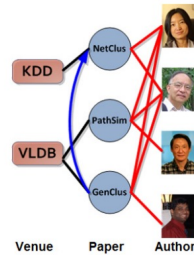
- One type of nodes
- One type of edges
- E.g. contact network



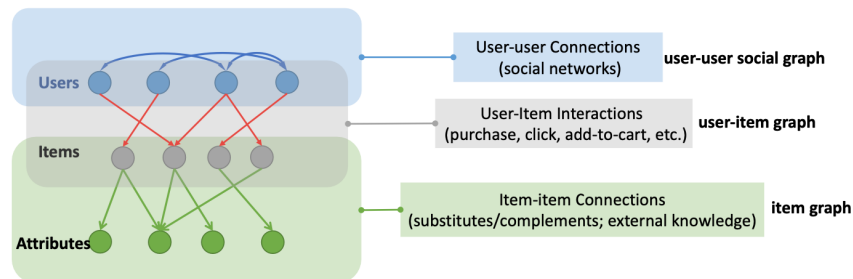
Homogeneous graphs

- **Heterogeneous graphs**

- Multiple types of nodes
- Multiple types of edges
- E.g. recommendation systems

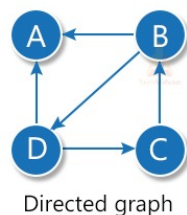


Heterogeneous graphs



Representation of a graph

- **Adjacency matrix representation**

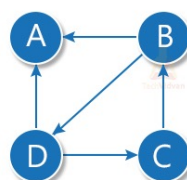


Directed graph

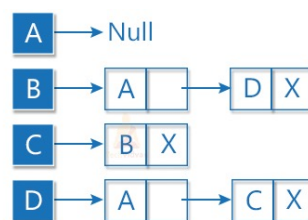
	A	B	C	D
A	0	0	0	0
B	1	0	0	1
C	0	1	0	0
D	1	0	1	0

Adjacency matrix

- **Adjacency list representation**



Directed graph



Adjacency List

Representation of a graph

- Adjacency matrix representation
 - Directly computable
 - Need $|V|^2$ space
- Adjacency list representation
 - Only need $|E|$ space
 - Not easy to compute

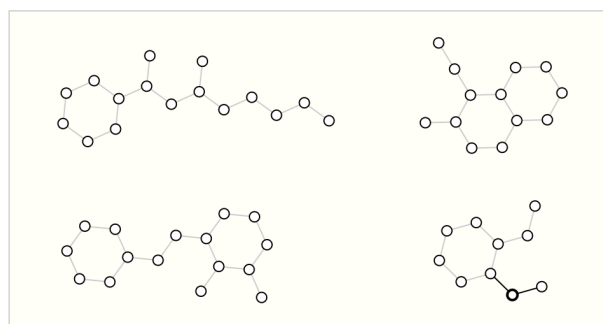
Representation of a graph

- Adjacency matrix representation
 - Directly computable
 - Need $|V|^2$ space
- Adjacency list representation
 - Only need $|E|$ space
 - Not easy to compute
- Latent representation
 - Need $|V| \times D$ space, where D is the dimension size
 - Computational friendly (also called embeddings)

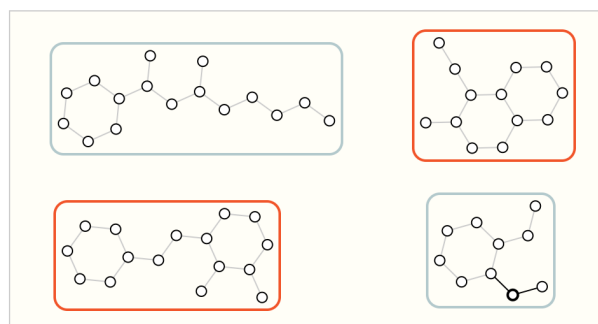
Downstream tasks for graphs

- Graph-level task**

- to predict a property of a (small) graph



Input: graphs

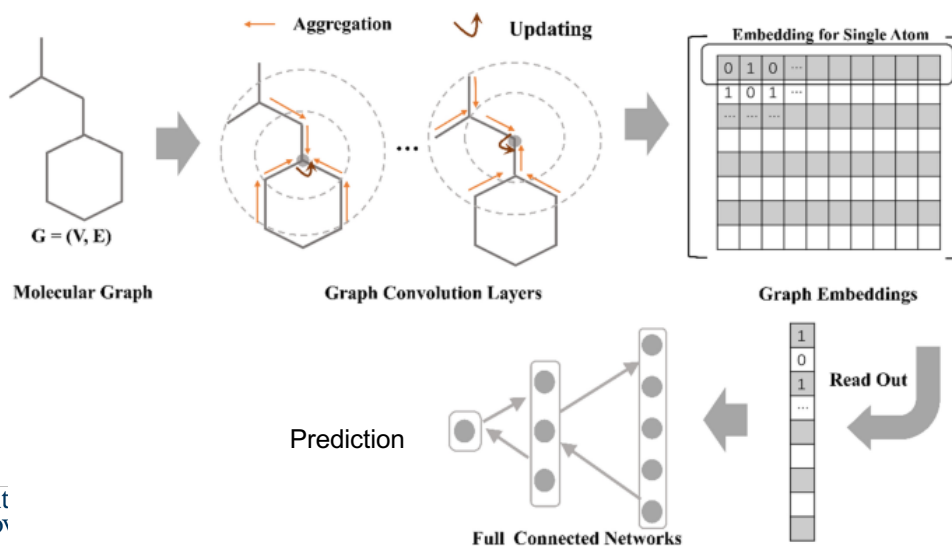


Output: labels for each graph, (e.g., "does the graph contain two rings?")

Downstream tasks for graphs

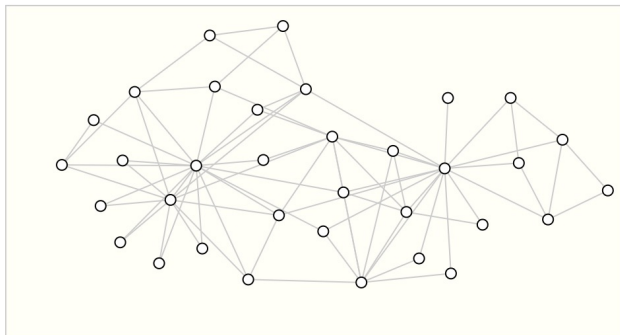
- Graph-level task**

- to predict a property of a (small) graph
- e.g. Molecule Property Prediction



Downstream tasks for graphs

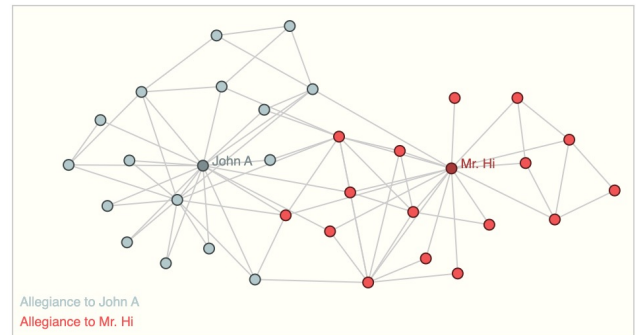
- **Graph-level task**
- **Node-level task**
 - to predict the property of each **node** within a (large) graph



Input: graph with unlabeled nodes



University of Glasgow | School of Computing Science



Output: graph node labels

23

Downstream tasks for graphs

- **Graph-level task**
- **Node-level task**
 - to predict the property of each **node** within a (large) graph
 - e.g. detecting political orientation using Facebook friend networks

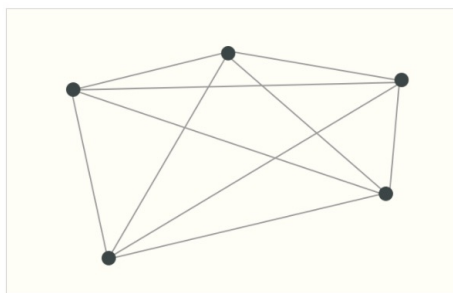


University of Glasgow | School of Computing Science

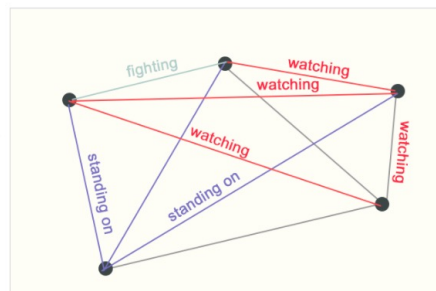
24

Downstream tasks for graphs

- **Graph-level task**
- **Node-level task**
- **Edge-level task**
 - to predict the property of each **edge** within a graph
 - e.g. link prediction



Input: fully connected graph, unlabeled edges

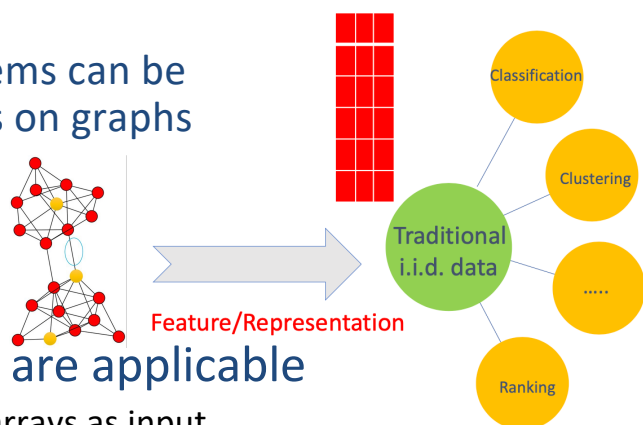


Output: labels for edges

25

Downstream tasks for graphs

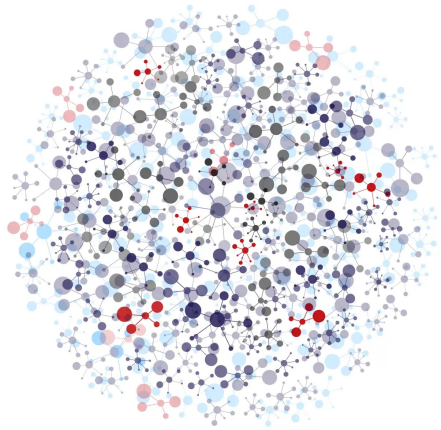
- Numerous real-world problems can be summarized as a set of tasks on graphs
 - Link prediction
 - Node Classification
 - Community Detection
 - Ranking
- Traditional ML solutions are applicable
 - Transfer graph into grid-like arrays as input (neighbors as feature)
- However, there are challenges
 - Arbitrary size and complex topological structure (i.e., no spatial locality like grids)
 - Often dynamic and have multimodal features
 - No fixed node ordering (not **permutation invariant**)



26

What are the requirements of graph databases

- **Flexibility:** Graph models adapt easily to changing applications.
- **Performance:** Graph databases need to handle complex relationships efficiently, improving query performance significantly.
- **Efficiency:** Graph queries are shorter and faster, leveraging persisted relationships between nodes, unlike relational databases that calculate joins during queries.



Graph Database Query Language (GQL)

What are GQLs?

- Query languages designed for graph data management.
- Define methods for extracting and retrieving graph-modeled data.
- Enable databases to communicate with clients via structured queries.

Popular GQLs:

- GraphQL
- Gremlin
- Cypher
- SPARQL
- AQL

Key Consideration:

- Each GQL has unique strengths and is suited for specific use cases.

Overview of Popular Graph Query Languages

GraphQL: A query language and runtime for **APIs** that allows clients to request only the data they need, making it flexible and efficient for hierarchical data retrieval.

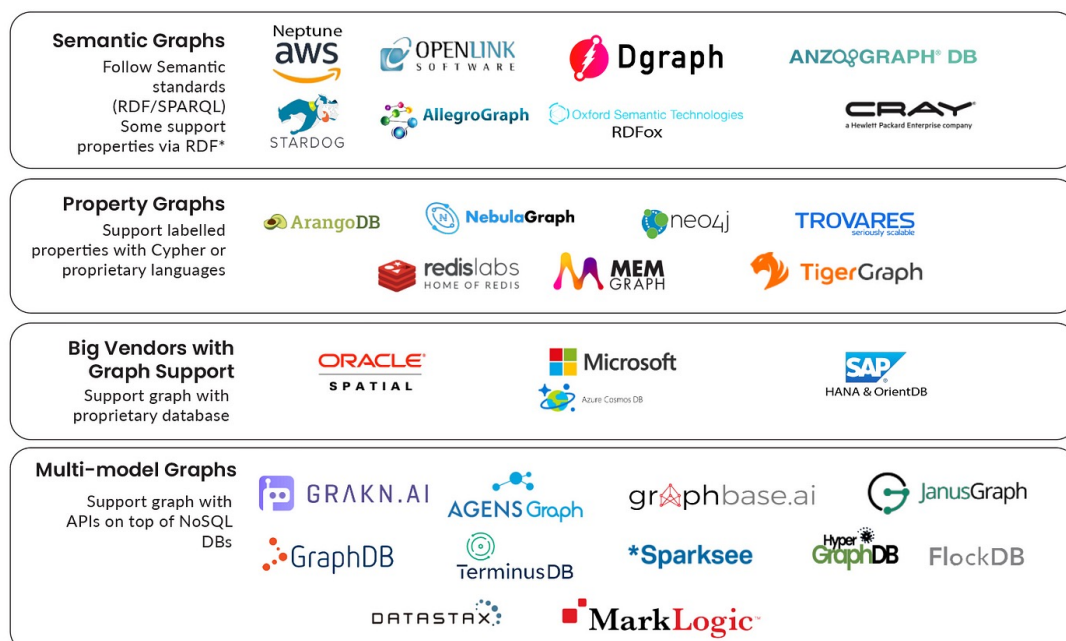
Gremlin: A **traversal-based** query language for graph databases that focuses on navigating and analysing graph structures.

Cypher: A declarative query language for **Neo4j** that simplifies querying, updating, and manipulating property graph data using pattern-matching syntax.

SPARQL: A query language for **RDF data** that enables querying and manipulation of semantic web datasets using triple patterns.

AQL (ArangoDB Query Language): A **SQL-like query language** for ArangoDB, supporting document, graph, and key-value data retrieval with seamless multi-model support.

The graph database market



Choosing the Right GQL

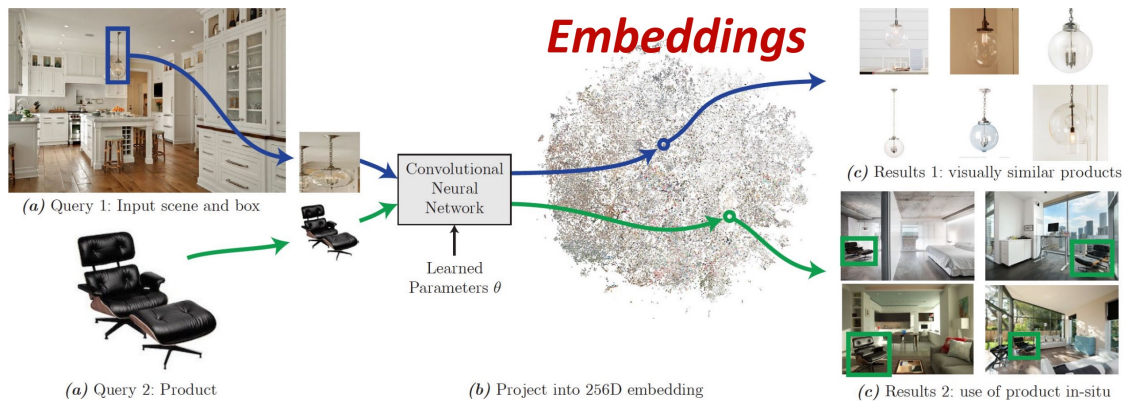
Use Cases:

- **GraphQL:** Best for complex systems and microservices.
 - **Gremlin:** Ideal for graph traversals and path-oriented queries.
 - **Cypher:** Suited for data analytics and app development.
 - **SPARQL:** Excels at data integration across intersecting domains.
 - **AQL:** Designed for scalable architectures and robust applications.
-
- GQLs offer diverse capabilities tailored to graph data operations.
 - Selection depends on specific data and application requirements.

Vector Database Systems

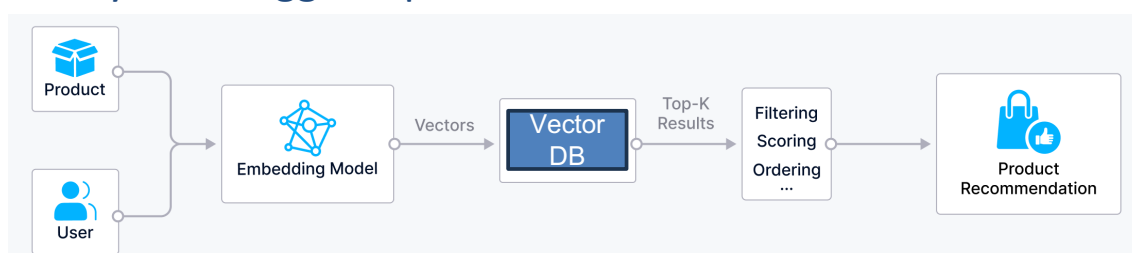
The Emerge of AI & Embeddings

- Used by search engines, recommender systems, personalized ads, etc.



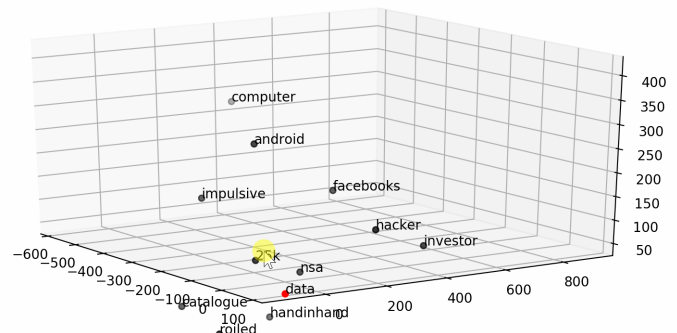
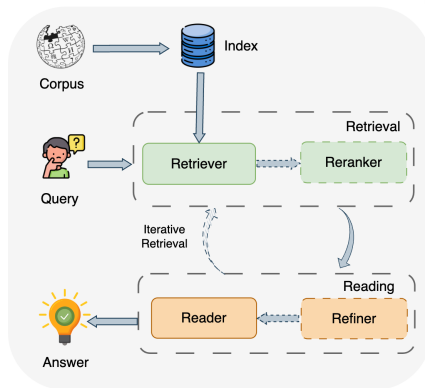
Vector Databases for Recommendation

- Users' purchase behaviours and product data are converted into embeddings via an RecSys model.
- These embeddings are stored and retrieved using a Vector DB.
- The Vector DB retrieves the Top-K most relevant results by comparing spatial distance between user and product vectors.
- After filtering, scoring, and ordering, the recommender system suggests products of interest to the user.



Retrieval Augmented Generation

- RAG models introduce a *retriever* model to retrieve relevant information from *external knowledge sources* to augment LLMs



Approximate K Nearest Neighbor (AKNN) Search

- Given a query vector q , find k vectors $V = \{v_1, v_2, \dots, v_k\}$ in storage that are approximately nearest to q
- Distance measure? – Euclidian distance, cosine similarity, etc.
- The higher recall the better
 - Let ground truth: V^*
 - Recall = $|V \cap V^*| / |V^*|$

AKNN Algorithms

- Tree-based: KD-tree, R-tree
- Quantization-based: IVF_FLAT/SQ8/PQ
- Graph-based: HNSW, NSG, SSG
- Locality sensitive hashing (LSH)

AKNN Libraries from AI Community

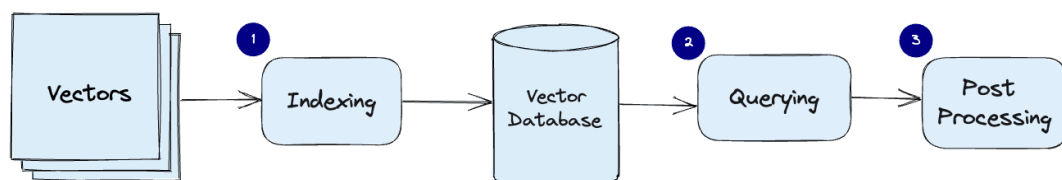
- Facebook **Faiss**, Microsoft **SPTAG**, Spotify **Annoy**, etc.
 - Implement various AKNN algorithms
- Pros: computation optimize
 - Support SIMD instructions
 - Faiss even supports GPU acceleration

AKNN Libraries from AI Community

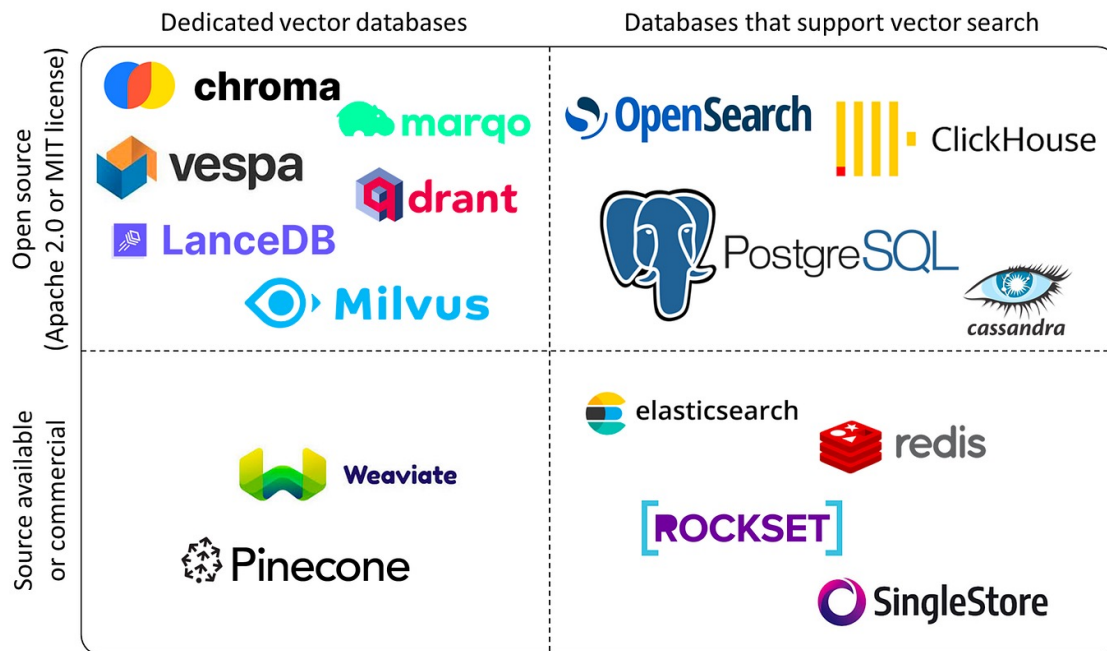
- Facebook **Faiss**, Microsoft **SPTAG**, Spotify **Annoy**, etc.
 - Implement various AKNN algorithms
- Cons:
 - Assume memory storage only
 - No support for dynamic data (updates/deletes)
 - No attribute filtering (e.g., “100 < price < 200”)

A common pipeline for a vector database

1. **Indexing:** The vector database indexes vectors using an algorithm such as PQ, LSH, or HNSW (more on these below). This step maps the vectors to a data structure that will enable faster searching.
2. **Querying:** The vector database compares the indexed query vector to the indexed vectors in the dataset to find the nearest neighbors (applying a similarity metric used by that index)
3. **Post Processing:** In some cases, the vector database retrieves the final nearest neighbors from the dataset and post-processes them to return the final results. This step can include re-ranking the nearest neighbors using a different similarity measure.



The landscape of vector databases



Summaries

- SQL databases struggle with scalability, schema flexibility, and handling unstructured data.
- NoSQL databases cater to semi-structured data types: key-value, wide column, document (tree), graph or vector.
- GQLs offer diverse capabilities tailored to graph data operations. But their selection depends on specific data and application requirements.
- A vector database efficiently stores, indexes, and retrieves high-dimensional vector embeddings for similarity search and ML/AI applications.

Do make use of **Evasys** to share
your valuable feedback with us!

Thank you!